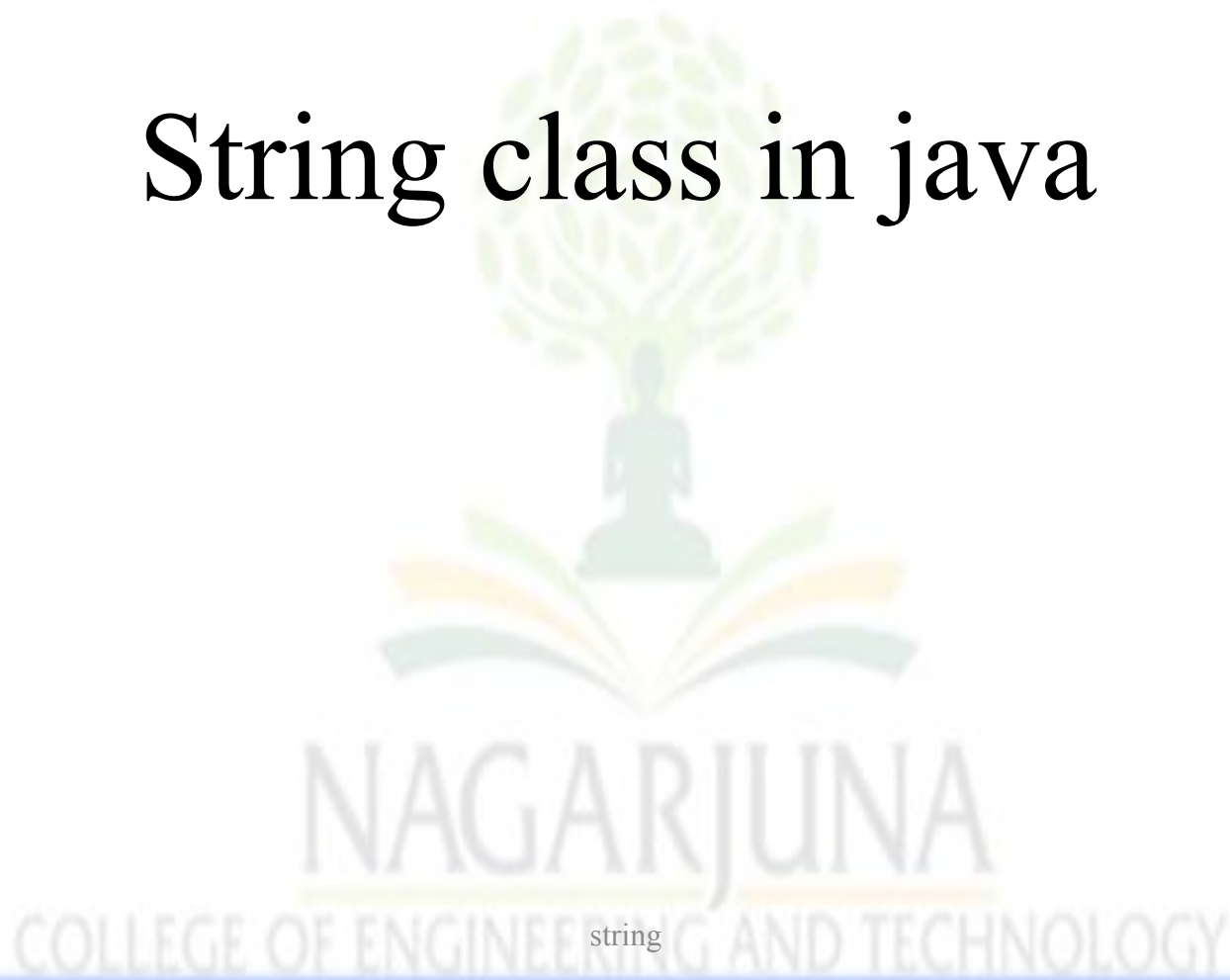


String class in java



string

Strings

- Java string is a sequence of characters. They are objects of type String.
- Once a String object is created it cannot be changed. **Strings are Immutable.**
- To perform **changes in original strings** use the class called **StringBuffer**.
- **String and StringBuffer classes are declared final**, so there cannot be subclasses of these classes.
- The default constructor creates an empty string.

String s = new String();

Creating Strings

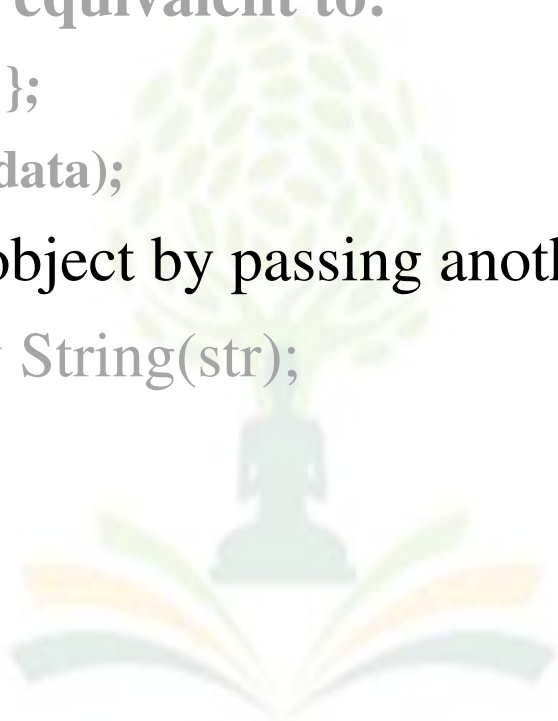
◎ `String str = "abc";` is equivalent to:

`char data[] = {'a', 'b', 'c'};`

`String str = new String(data);`

◎ Construct a string object by passing another string object.

`String str2 = new String(str);`



NAGARJUNA

COLLEGE OF ENGINEERING AND TECHNOLOGY

string

String METHODS

- ◎ The `length()` method returns the length of the string.
Eg: `System.out.println("Hello".length());` // prints 5
- ◎ The `+` operator is used to concatenate two or more strings.
Eg: `String myname = "Harry"`
`String str = "My name is" + myname + ".";`
- ◎ For string concatenation the Java compiler converts an operand to a `String` whenever the other operand of the `+` is a `String` object.

- public char **charAt**(int index)
 - Returns the character at the specified index. An index ranges from 0 to length() - 1. The first character of the sequence is at index 0, the next at index 1, and so on, as for array indexing.

```
char ch;
```

```
ch = "abc".charAt(1); // ch = "b"
```

- ◎ **equals()** - Compares the invoking string to the specified object. The result is true if and only if the argument is not null and is a String object that represents the same sequence of characters as the invoking object.

public boolean **equals**(Object anObject)

- ◎ **equalsIgnoreCase()**- Compares this String to another String, ignoring case considerations. Two strings are considered equal ignoring case if they are of the same length, and corresponding characters in the two strings are equal ignoring case.

public boolean **equalsIgnoreCase**(String anotherString)

- **startsWith()** – Tests if this string starts with the specified prefix.

public boolean **startsWith**(String prefix)

“Figure”.startsWith(“Fig”); // true

- **endsWith()** - Tests if this string ends with the specified suffix.

public boolean **endsWith**(String suffix)

“Figure”.endsWith(“re”); // true

- **compareTo()** - Compares two strings.
 - The result is a negative integer if this String object lexicographically precedes the argument string.
 - The result is a positive integer if this String object lexicographically follows the argument string.
 - The result is zero if the strings are equal.
 - compareTo returns 0 exactly when the equals(Object) method would return true.

public int **compareTo**(String anotherString)

public int **compareToIgnoreCase**(String str)

- **indexOf** – Searches for the **first occurrence** of a character or **substring**. Returns -1 if the character does not occur.

public int **indexOf**(String str) - Returns the index within this string of the first occurrence of the specified substring.

```
String str = "How was your day today?";  
str.indexOf('t');  
str("was");
```

- **lastIndexOf()** –Searches for the **last occurrence** of a character or **substring**. The methods are similar to **indexOf()**.

- **substring()** - Returns a new string that is a **substring of this string**. The substring begins with the character at the specified index and extends to the end of this string.

public String **substring**(int beginIndex)

Eg: "unhappy".substring(2) returns "happy"

- public String **substring**(int beginIndex, int endIndex)

Eg: "smiles".substring(1, 5) returns "mile"

String METHODS

Method call	Meaning
S2=s1.toLowerCase()	Convert string s1 to lowercase
S2=s1.toUpperCase()	Convert string s1 to uppercase
S2=s1.repalcce('x', 'y')	Replace occurrence x with y
S2=s1.trim()	Remove whitespaces at the beginning and end of the string s1
S1.equals(s2)	If s1 equals to s2 return true
S1.equalsIgnoreCase(s2)	If s1==s2 then return true with irrespective of case of charecters
S1.length()	Give length of s1
S1.CharAt(n)	Give nth character of s1 string
S1.compareTo(s2)	If s1<s2 -ve no If s1>s2 +ve no If s1==s2 then 0
S1.concat(s2)	Concatenate s1 and s2
S1.substring(n)	Give substring staring from nth character

String Operations



NAGARJUNA

COLLEGE OF ENGINEERING AND TECHNOLOGY

string

String Operations

concat() - Concatenates the specified string to the end of this string.

If the length of the argument string is 0, then this String object is returned.

Otherwise, a new String object is created, containing the invoking string with the contents of the str appended to it.

```
public String concat(String str)  
"to".concat("get").concat("her")
```

returns "together"

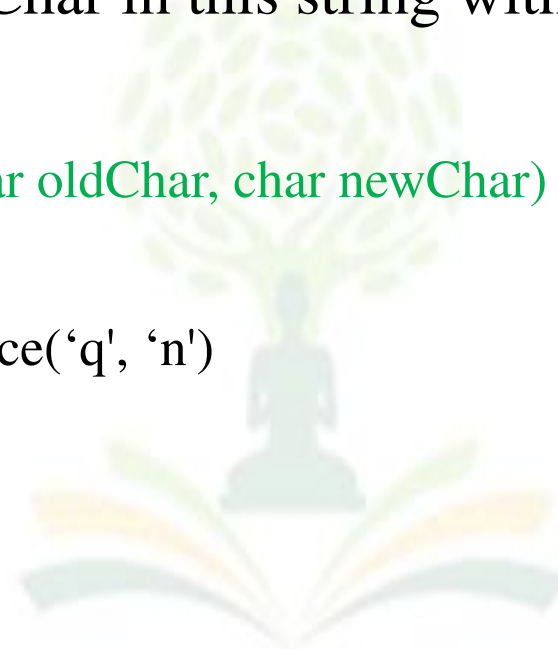
String Operations

- `replace()`- Returns a new string resulting from replacing all occurrences of `oldChar` in this string with `newChar`.

`public String replace(char oldChar, char newChar)`

`“iam aq iqdiaq ” . replace(‘q’, ‘n’)`

returns `“I am an indian”`



NAGARJUNA

COLLEGE OF ENGINEERING AND TECHNOLOGY

string

String Operations

- **trim()** - Returns a copy of the string, with leading and trailing whitespace omitted.

```
public String trim()
String s = "    Hi Mom! "
    s.trim();
S = "Hi Mom!"
```

- **valueOf()** – Returns the string representation of the char array argument.

```
public static String valueOf(char[] data)
```

String Operations

- **toLowerCase():** Converts all of the characters in a String to lower case.
- **toUpperCase():** Converts all of the characters in this String to upper case.

public String **toLowerCase()**

public String **toUpperCase()**

Eg: "HELLO THERE".toLowerCase();

"hello there".toUpperCase();

StringBuffer

- A StringBuffer is like a String, but can be modified.
- The length and content of the StringBuffer sequence can be changed through certain method calls.
- StringBuffer defines three constructors:
 - StringBuffer()
 - StringBuffer(int size)
 - StringBuffer(String str)

StringBuffer Operations

- The principal operations on a StringBuffer are the append and insert methods, which are overloaded so as to accept data of any type.

Here are few append methods:

`StringBuffer append(String str)`

`StringBuffer append(int num)`

- The append method always adds these characters at the end of the buffer.

StringBuffer Operations

- Insert method adds the characters at a specified point.

Here are few insert methods:

```
StringBuffer insert(int index, String str)
```

```
StringBuffer append(int index, char ch)
```

Index specifies at which point the string will be inserted into the invoking StringBuffer object.

StringBuffer Operations

- **delete()** - Removes the characters in a substring of this StringBuffer. The substring begins at the specified start and extends to the character at index end - 1 or to the end of the StringBuffer if no such character exists. If start is equal to end, no changes are made.

```
public StringBuffer delete(int start, int end)
```

StringBuffer Operations

- **replace()** - Replaces the characters in a substring of this StringBuffer with characters in the specified String.

```
public StringBuffer replace(int start, int end,  
    String str)
```

- **substring()** - Returns a new String that contains a subsequence of characters currently contained in this StringBuffer. The substring begins at the specified index and extends to the end of the StringBuffer.

```
public String substring(int start)
```

StringBuffer Operations

- **reverse()** - The character sequence contained in this string buffer is replaced by the reverse of the sequence.

```
public StringBuffer reverse()
```

- **length()** - Returns the length of this string buffer.

```
public int length()
```

- **setLength()** - Sets the length of the StringBuffer.

```
public void setLength(int newLength)
```

StringBuffer Operations

- **capacity()** - Returns the current capacity of the String buffer. The capacity is the amount of storage available for newly inserted characters.

```
public int capacity()
```

- **charAt()** - The specified character of the sequence currently represented by the string buffer, as indicated by the index argument, is returned.

```
public char charAt(int index)
```

StringBuffer Methods

Methods	Meaning
S1.setCharAt(n, 'x')	Modify the nth character to x
S1.append(s2)	Append s2 string at the end of s1
S1.insert(n,s2)	Insert string s2 in s1 at position n
S1.setLength(n)	Set length of string s1 to n



Examples: StringBuffer

```
StringBuffer sb = new StringBuffer("Hello");
```

```
sb.length(); // 5
```

```
sb.capacity(); // 21 (16 characters room is added if no size is  
specified)
```

```
sb.charAt(1); // e
```

```
sb.setCharAt(1,'i'); // Hillo
```

```
sb.setLength(2); // Hi
```

```
sb.append("l").append("l"); // Hill
```

```
sb.insert(0, "Big "); // Big Hill
```

```
sb.replace(3, 11, " "); // Big
```

```
sb.reverse(); // gib
```

vectors

- Vector class is present in java.util language package.
- It Creates generic dynamic array which hold object of any type any number.
- The object do not have to be homogeneous
- In vector class only objects are stored no type is stored.
 - `Vector a = new Vector();`
 - `Vector a = new Vector(3);`

Vector methods

Method	Meaning
a.addElement(item)	Add item specified to list at the end
a.elementAt(10)	Give name of 10 object
a.Size()	Number objects present
a.removeElement(item)	Remove specified item form list a
a.removeEelementAt(n)	Remove item stored in n position
a.reomveAllElements()	Remove all elements from list
a.insertElementAt(item,n)	Insert item at n position

Difference between arraylist and vector

ArrayList	Vector
1) ArrayList is not synchronized.	Vector is synchronized.
2) ArrayList increments 50% of current array size if number of element exceeds from its capacity.	Vector increments 100% means doubles the array size if total number of element exceeds than its capacity.
3) ArrayList is not a inheritance class, it is introduced in JDK 1.2.	Vector is a legacy class.
4) ArrayList is fast because it is non-synchronized.	Vector is slow because it is synchronized i.e. in multithreading environment, it will hold the other threads in runnable or non-runnable state until current thread releases the lock of object.
5) ArrayList uses Iterator interface to traverse the elements.	Vector uses Enumeration interface to traverse the elements. But it can use Iterator also.

Wrapper class

- To handle primitive data types java support it by using wrapper class.
- **java** provides the mechanism *to convert primitive into object and object into primitive*.
- **autoboxing** and **unboxing** feature converts primitive into object and object into primitive automatically.
- The automatic conversion of primitive into object is known as autoboxing and vice-versa unboxing.

Example of wrapper class

```
public class WrapperExample1{  
    public static void main(String args[]){  
        //Converting int into Integer  
        int a=20;  
        Integer i=Integer.valueOf(a);//converting int into Integer  
        Integer j=a;//autoboxing, now compiler will write Integer.valueOf(a) internally  
  
        System.out.println(a+" "+i+" "+j);  
    }  
}
```

Output:

20 20 20

Enumerated type

- Java allows enumerated type with help of enum keyword

Public class days

```
{  
    Public static final int day_Sunday=0;  
    Public static final int day_monday=0;  
    Public static final int day_tuesday=0;  
    Public static final int day_wednesday=0;  
    Public static final int day_thursday=0;  
    Public static final int day_friday=0;  
    Public static final int day_Saturday=0;  
}
```

example

```
Public class enum
{
Enum day {
Sunday,Monday,Tuesday,Wednes
day,Thursday,Friday,satarday}
Public static void main(String
args[])
{
for(day d:day.values())
{
Weekend(d);
} }
}
```

```
Private weekend(day d)
{
If(d.equals(day.Sunday)
System.out.println("holiday+d);
Else
System.out.println("working
days+d);
}
}
```

NAGARJUNA

COLLEGE OF ENGINEERING

st

For more detail contact us